WEEK 2 · LESSON 9 OF 24 · HANDOUT

Structured Output and Tool Use

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=jOEdb9HnwsQ — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyOhio 2025: Demystifying AI Agents with Python Code — https://www.youtube.com/watch?v=jOEdb9HnwsQ

🎓 **Go deeper:** PyData Virginia 2025: Mastering LLMs — from Prompt Engineering to Agentic AI — https://www.youtube.com/watch?v=jmwLzX_ltbQ

Your goal in this lesson: Get reliable structured data (JSON) out of an LLM, validate it with Pydantic, and let the model call your own Python functions as tools.

Text is for people; programs need structure

Your app needs fields it can store and act on — a category, an amount, a phone number — not a paragraph. Two techniques:

1. **Ask for JSON and validate it.**
2. **Use a tool definition (a JSON schema) to force the structure.**

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

Technique 1 — Ask for JSON, then validate

```
import json
from pydantic import BaseModel, ValidationError, Field
from typing import Literal

class Ticket(BaseModel):
    category: Literal["payment", "network", "account"]
    urgent: bool
    summary: str = Field(max_length=120)

def extract_ticket(message, attempts=2):
    prompt = (
        "Read the customer message and reply with JSON only, no other text, with keys: "
        '"category" (payment, network or account), "urgent" (true or false), '
        '"summary" (under 120 characters).\n\n'
        f"<message>{message}</message>"
    )
    for attempt in range(attempts):
        raw = ask(prompt, max_tokens=200)
        try:
            return Ticket.model_validate(json.loads(raw))
        except (json.JSONDecodeError, ValidationError) as error:
            print(f"attempt {attempt + 1} failed validation: {type(error).__name__}")
            prompt += "\nYour last reply was not valid. Reply with the JSON object only."
    return None

ticket = extract_ticket("URGENT: money was deducted twice when I paid my rent!")
print(ticket)
```

Pydantic checks every field: the category must be one of the three words, `urgent` must be true or false, the summary must be short. If anything is wrong, we retry once with a reminder, and give up safely (return `None`) instead of crashing.

Technique 2 — Force the shape with a tool

The API lets you describe a "tool" with a **JSON schema** and require the model to use it. The model then returns arguments that follow the schema. Pydantic can even write the schema for you:

```
ticket_tool = {
    "name": "save_ticket",
    "description": "Save a classified customer support ticket.",
    "input_schema": Ticket.model_json_schema(),
}
response = client.messages.create(
    model=MODEL, max_tokens=300,
    tools=[ticket_tool],
    tool_choice={"type": "tool", "name": "save_ticket"},      # must use this tool
    messages=[{"role": "user", "content": "My SIM card is blocked, I cannot receive my
salary!"}],
)
tool_call = next(block for block in response.content if block.type == "tool_use")
print(tool_call.input)
print(Ticket.model_validate(tool_call.input))
```

This is usually more reliable than asking for JSON in words. Still validate: your code, not the model, is responsible for what gets saved.

Tool use: letting the model call your functions

Tools are not only for structure. You can give the model **real functions** — check a balance, look up an exchange rate, search a database. The model **decides** when a tool is needed and with what arguments; **your code runs it** and sends back the result; the model then writes the final answer.

```
# Two "real" functions (here with practice data)
BALANCES = {"0788123456": 45_300, "0722000111": 1_250}
RATES_TO_RWF = {"USD": 1_400, "EUR": 1_500, "KES": 10.8}       # practice rates, not live data

def get_balance(phone: str) -> str:
    return f"{BALANCES.get(phone, 0):,} RWF" if phone in BALANCES else "unknown number"

def convert_to_rwf(amount: float, currency: str) -> str:
    rate = RATES_TO_RWF.get(currency.upper())
    return f"{amount * rate:,.0f} RWF" if rate else "unknown currency"

TOOLS = [
    {"name": "get_balance", "description": "Get the mobile-money balance of a phone number.",
     "input_schema": {"type": "object", "properties": {"phone": {"type": "string"}},
"required": ["phone"]}},
    {"name": "convert_to_rwf", "description": "Convert an amount of a foreign currency to
Rwandan francs.",
     "input_schema": {"type": "object",
                      "properties": {"amount": {"type": "number"}, "currency": {"type":
"string", "enum": ["USD", "EUR", "KES"]}},
                      "required": ["amount", "currency"]}},
]
FUNCTIONS = {"get_balance": get_balance, "convert_to_rwf": convert_to_rwf}
```

The tool loop

```
def run_with_tools(question, max_rounds=5):
    messages = [{"role": "user", "content": question}]
    for _ in range(max_rounds):
        response = client.messages.create(model=MODEL, max_tokens=500, tools=TOOLS,
messages=messages)
        if response.stop_reason != "tool_use":
            return "".join(b.text for b in response.content if b.type == "text")
        messages.append({"role": "assistant", "content": response.content})
        results = []
        for block in response.content:
            if block.type == "tool_use":
                output = FUNCTIONS[block.name](**block.input)        # YOUR code runs the tool
                print(f"  tool {block.name}({block.input}) -> {output}")
                results.append({"type": "tool_result", "tool_use_id": block.id, "content":
output})
        messages.append({"role": "user", "content": results})
    return "Stopped: too many tool rounds."

print(run_with_tools("What is the balance of 0788123456?"))
```

Step by step: the model replies with `stop_reason == "tool_use"` and the tool name and arguments; your code runs the function; you send a `tool_result` back; the model writes a normal answer using the result. `max_rounds` stops endless loops.

Safety rules for tools

- The model only **asks**; **your code decides** whether to run the tool. Check arguments (is this phone number allowed for this user?).
- Give read-only tools first. Anything that moves money or deletes data needs a **human confirmation** step.
- Log every tool call.

Key terms

- **Structured output** — data in a fixed format such as JSON.
- **JSON schema** — a description of the fields and types allowed.
- **Pydantic** — a Python library that validates data against a model.
- **Tool use (function calling)** — the model requests a function call; your code runs it.
- **Tool result** — the output you send back for the model to use.

Remember this: Validate every structured output with code, use tool schemas to force the shape, and remember that in tool use the model only asks — your program decides and acts.

Name: _____ Date: _____

Activity

Task 1

Add a field `amount_rwf: int | None` to `Ticket` and test it on messages with and without an amount.

Task 2

Ask "How much is 50 US dollars in Rwandan francs?" with `run_with_tools`. Which tool is used?

Task 3

Add a tool `report_fraud(phone, description)` that only **prints** a report. When should it require a person to confirm?

Task 4

Make `get_balance` refuse numbers that do not belong to the logged-in user. Where does that check belong — in the prompt or in the code?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why validate an LLM's JSON with Pydantic?
   - A. To make it longer
   - B. Pydantic calls the API
   - C. It is required by JSON
   - D. Because the model can return wrong or missing fields
2. What does tool_choice={'type': 'tool', 'name': 'save_ticket'} do?
   - A. Disables tools
   - B. Deletes the ticket
   - C. Forces the model to answer by using that tool
   - D. Runs the tool automatically
3. In tool use, who actually runs the function?
   - A. Nobody
   - B. The model
   - C. The user
   - D. Your code
4. What stop_reason means the model wants a tool?
   - A. end_turn
   - B. stop_sequence
   - C. tool_use
   - D. max_tokens
5. Why limit the tool loop with max_rounds?
   - A. It makes answers longer
   - B. Tools only work once
   - C. To avoid endless loops and runaway costs
   - D. To save the API key
6. Which tool most needs a human confirmation step?
   - A. Send money to another account
   - B. Check a balance
   - C. Convert currency
   - D. Look up a holiday date

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.